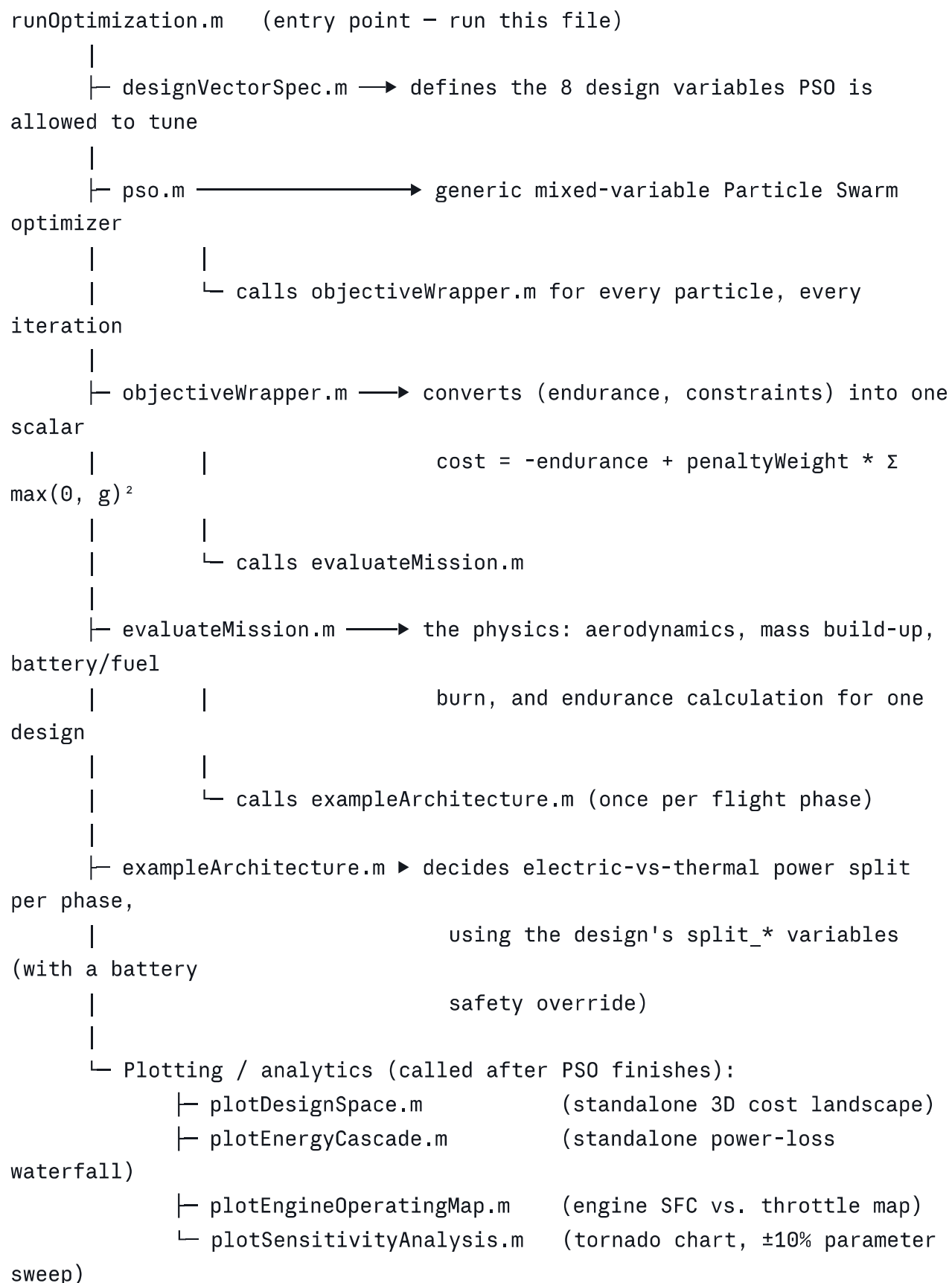


Hybrid-Electric Propulsion Sizing & Power-Split Optimizer

A MATLAB toolchain that uses **Particle Swarm Optimization (PSO)** to size a parallel-hybrid propulsion system (turboshaft engine + battery + electric motors) and simultaneously decide how electric power should be blended in with engine power across every phase of a flight mission — takeoff, climb, cruise, loiter, descent — in order to **maximize flight endurance** subject to real aerodynamic, mass, and power constraints.

1. How the pieces fit together



One call = one design evaluated. PSO never touches physics directly — it only ever calls `objFcn(designStruct)` and gets a number back. Everything about the aircraft lives in `evaluateMission.m`; everything about the optimizer lives in `pso.m`. This separation means you can swap in a smarter mission model or a different optimizer without touching the other.

2. File-by-file reference

`runOptimization.m` — Run this one.

The top-level script. It wires up the design spec, the architecture function, the mission model, and PSO's settings, runs the optimization, prints the best design and its endurance, and then generates a full suite of figures via local helper functions defined at the bottom of the same script (`plotCoreDashboard`, `plotDesignSpaceTopology`, `plotAerodynamicPolarMap`, `plotOptimizedArchitecture`, plus local versions of the engine-map and energy-cascade plots).

`designVectorSpec.m`

Declares the 8 variables PSO searches over, each with a name, bounds, and type:

Variable	Bounds	Type	Meaning
<code>engineSize_kW</code>	20 - 150	continuous	Turboshaft rated power
<code>batteryCap_kWh</code>	5 - 60	continuous	Battery energy capacity
<code>numMotors</code>	1 - 4	integer	Number of electric motors
<code>split_takeoff</code>	0 - 1	continuous	Electric power fraction during takeoff
<code>split_climb</code>	0 - 1	continuous	Electric power fraction during climb
<code>split_cruise</code>	0 - 1	continuous	Electric power fraction during cruise
<code>split_loiter</code>	0 - 1	continuous	Electric power fraction during loiter
<code>split_descent</code>	0 - 1	continuous	Electric power fraction during descent

Add or remove rows freely — `pso.m` never hardcodes the variable count or order.

`pso.m` — the optimizer engine

A generic, mixed-variable Particle Swarm Optimizer. Handles continuous, integer, and categorical variables, linear inertia-weight decay, velocity clamping, stall-based early stopping, optional `parfor` evaluation, and reproducible runs via an RNG seed. It knows nothing about propulsion — it only needs `objFcn`, `varSpec`, and `opts`. Returns the best design (`bestDesign`), its cost (`bestFval`), and the convergence history (`history`).

`objectiveWrapper.m` — physics → single number

Bridges the mission model and the optimizer. Calls `evaluateMission`, then converts `(endurance_s, g)` into a minimizable scalar using a quadratic exterior penalty:

```
cost = -endurance_s + penaltyWeight * sum(max(0, g).^2)
```

Raise `penaltyWeight` if PSO keeps returning infeasible designs; lower it if the search gets stuck refusing to explore near the constraint boundary.

`evaluateMission.m` — the physics core

The most important file to understand and extend. For a given design:

1. **Mass build-up** — sizes engine/motor/battery mass from specific-power assumptions, then derives how much fuel mass is left under a fixed 1000 kg MTOW.
2. **Aerodynamics** — computes required power at each flight phase from altitude, speed, wing area, aspect ratio, Oswald efficiency, and zero-lift drag.
3. **Power routing** — asks the architecture function how to split each phase's power between engine and battery, then tracks fuel burn (via a throttle-dependent SFC curve) and battery state of charge.
4. **Endurance** — treats takeoff/climb/loiter/descent as fixed-duration phases and solves for how long cruise can be sustained before either the fuel or the battery runs out.
5. **Constraints returned** (`g`, all "violated if > 0"):
 - `g1` — empty mass + payload exceeds MTOW (no fuel margin left)
 - `g2` — reserved for state-of-charge (currently folded into endurance calc)
 - `g3` — engine power shortfall (thermal demand exceeds rated engine power)
 - `g4` — insufficient fuel for the fixed-duration phases

`exampleArchitecture.m` — the power-split policy

The "brain" that decides, for each flight phase, what fraction of required power comes from the battery vs. the engine. In this version, PSO directly controls that fraction (the `split_*` design variables) — this is what makes the power-management strategy itself part of the optimization, not just the hardware sizing. Includes one hard safety rule: if battery state of charge drops below 20%, it forces engine-only operation regardless of what the optimizer requested.

Plotting utilities

File	Purpose
<code>plotDesignSpace.m</code>	3D surface of endurance vs. engine size & battery capacity, with the constraint boundary and the optimum marked
<code>plotEnergyCascade.m</code>	Waterfall chart of power losses from fuel/battery input down to useful aerodynamic thrust for one phase
<code>plotEngineOperatingMap.m</code>	SFC-vs-throttle curve with each mission phase's operating point overlaid
<code>plotSensitivityAnalysis.m</code>	Tornado chart showing how ±10% changes in key physical parameters shift endurance

3. Running it

```
matlab
runOptimization
```

That's the entire workflow — it runs PSO end-to-end and produces all figures. Typical console output:

```
Iter 10 | best cost = -18734.2 | stall = 2
...
Maximized Flight Endurance: 6.83 Hours
Total Algorithm Execution Time: 4.12 Seconds
```

To tune the search itself, edit the `opts` struct near the top of `runOptimization.m` (swarm size, iteration cap, inertia/cognitive/social coefficients, penalty weight, RNG seed).

4. A few things worth knowing before you extend this

- `plotDesignSpace.m` and `plotSensitivityAnalysis.m` are standalone utilities — they are *not* called anywhere inside `runOptimization.m`. `runOptimization.m` uses its own local `plotDesignSpaceTopology` instead, which does the same 3D-landscape job. Call the standalone versions manually if you want them (e.g. `plotDesignSpace(varSpec, bestDesign, history, missionFcn, archFcn)`).
- `plotEnergyCascade` and `plotEngineOperatingMap` exist in two places — as standalone files (this folder) and as local functions of the same name inside `runOptimization.m`. The local versions are what actually run when you execute `runOptimization.m`, and they take slightly different arguments (`archFcn` instead of `phaseName` for the cascade plot). Worth reconciling into one version if you plan to keep maintaining both.
- `plotSensitivityAnalysis.m` currently uses hard-coded illustrative numbers rather than a live finite-difference sweep of `evaluateMission.m` — useful as a template, but replace the `sensitivity_low` / `sensitivity_high` arrays with real perturbation results before trusting the chart.
- **To add a new design variable** (e.g. a new architecture parameter), add one line to `designVectorSpec.m` and reference `designStruct.<name>` in `exampleArchitecture.m` or `evaluateMission.m` — no change needed in `pso.m` or `objectiveWrapper.m`.
- **To try a different power-management policy**, write a new function with the same signature as `exampleArchitecture.m` (`pShare = myPolicy(designStruct, phaseName, missionState)`) and pass its handle into `objectiveWrapper` instead.